Cooling the Foundry Floor

A robotics team runs a square foundry floor divided into an `n x n` grid of tiles. Each tile `(i, j)` has been heated to a temperature given by `grid[i][j]`.

A cooling system is switched on. After it runs for `t` seconds, every tile whose temperature is at most `t` has cooled down enough to be safe to stand on. A tile whose temperature is strictly greater than `t` is still too hot to touch.

An inspection robot starts on the top-left tile `(0, 0)`. Each second the robot can step to a tile that shares an edge with its current tile (up, down, left, or right), but only if **both** the tile it is leaving and the tile it is entering are already safe (temperature at most `t`). Once tiles are safe, the robot can move across any number of them instantly, so travel between safe tiles takes no time. The robot must always stay inside the grid.

Determine the earliest second `t` at which the robot can reach the bottom-right tile `(n - 1, n - 1)`.

Example Input

```
2
0 2
1 3
```

Output

```
3
```

Explanation

At second 0 the robot sits on tile `(0, 0)`. Both neighbouring tiles are still too hot, so it cannot move. Only once the cooling has run for 3 seconds are all tiles safe, and the robot can walk straight to `(1, 1)`.

Input Format

The first line contains a single integer `n`, the side length of the grid.

Each of the next `n` lines contains `n` space-separated integers; the `j`-th value on the `i`-th of these lines is `grid[i][j]`, the temperature of tile `(i, j)`.

Constraints

n == number of rows == number of columns

1 <= n <= 50

0 <= grid[i][j] < n^2

Every value grid[i][j] is distinct.

Output Format

Print a single integer: the earliest second at which tile `(n - 1, n - 1)` becomes reachable from tile `(0, 0)`.

Sample Input 0

```
1
0
```

Sample Output 0

```
0
```

Sample Input 1

```
2
0 2
1 3
```

Sample Output 1

```
3
```

Sample Input 2

```
2
0 1
3 2
```

Sample Output 2

```
2
```